MigrateCRM

Implementation Plan — Enterprise CRM for an Australian Registered Migration Agency

Version	2.0 (replaces v1 Next.js plan)
Date	2 November 2026
Backend	Python 3.12 · FastAPI · SQLAlchemy 2 · Alembic
Frontend	React 18 · TypeScript · Vite · TanStack Query · Tailwind CSS
Database	Neon (serverless PostgreSQL), AWS Sydney region
Jurisdiction	Australia (MARA / OMARA regulated agents)
Scope	Core CRM + Billing & Client Account + Client Portal + Reports

Contents

1. Goals and scope
2. Technology decisions
3. System architecture
4. Repository structure
5. Functional modules
6. Roles
7. Permission matrix and access rules
8. Data model
9. API design
10. Frontend design
11. Security
12. Australian compliance support
13. Neon database operations
14. Testing and quality
15. Deployment and environments
16. Delivery phases
17. Risks and open questions

1. Goals and scope

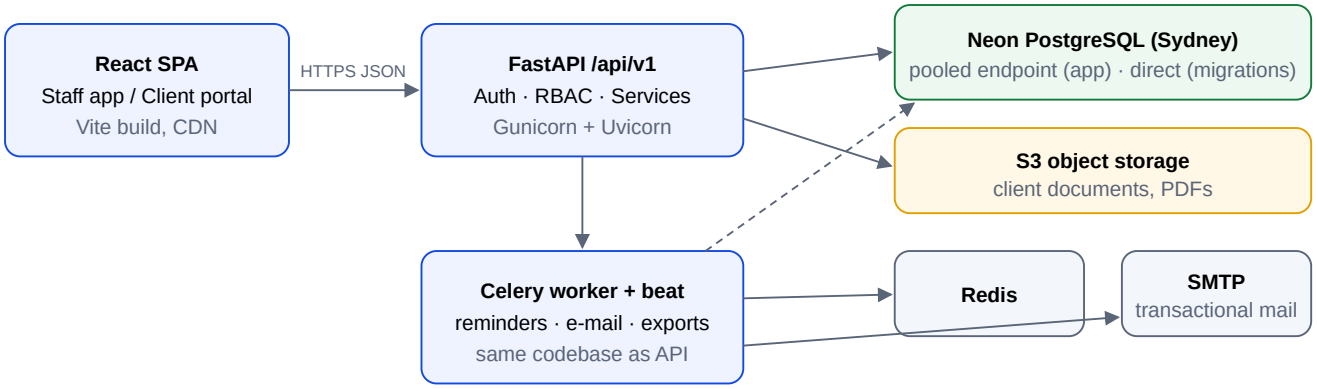
Build a production-grade CRM for an Australian migration agency, modelled on Salesforce concepts: **Leads** are converted into **Contacts** (clients), **Organisations** (accounts, e.g. employer sponsors) and **Matters** (visa cases, the equivalent of Opportunities/Cases). Every record has a Salesforce-style record page: highlights panel, stage **Path**, related lists and an activity timeline.

- **In scope:** leads, contacts, organisations, matters, tasks & activities, documents, billing (service agreements, invoices, payments, client/trust account), client portal, reports with CSV export, administration, audit log, notifications.
- **Out of scope (v1):** direct lodgement into ImmiAccount (no public API), e-mail inbox sync, SMS gateway, accounting-package sync (Xero/MYOB), mobile apps. The design leaves extension points for these.
- **Quality bar:** role-based security enforced on the server, full audit trail, automated tests, typed API contract, reproducible deployments.

2. Technology decisions

Area	Decision	Reason
Backend framework	FastAPI on Python 3.12, served by Uvicorn (Gunicorn workers in production)	Typed, fast, automatic OpenAPI schema.
ORM / migrations	SQLAlchemy 2.0 (typed ORM) + Alembic	Mature, explicit transactions, versioned schema changes.
Validation	Pydantic v2 schemas for every request and response	One source of truth for API shapes.
Database	Neon serverless PostgreSQL 16+, project in `aws-ap-southeast-2` (Sydney)	Managed Postgres, branching for dev/preview, Australian data residency.
DB driver	psycopg 3 (`postgresql+psycopg://`), `sslmode=require`	Well supported by SQLAlchemy and Neon.
Background jobs	Celery + Redis (worker + beat scheduler)	E-mails, deadline / visa-expiry reminders, report exports.
File storage	S3-compatible object storage (AWS S3, Sydney region); local disk adapter for development	Durable, encrypted at rest, pre-signed downloads.
PDF generation	WeasyPrint (HTML templates → PDF)	Invoices and service agreements.
Passwords / MFA	Argon2id (`argon2-cffi`); TOTP MFA (`pyotp`)	Current best practice; MFA mandatory for staff.
Frontend	React 18 + TypeScript, Vite, React Router, TanStack Query, React Hook Form + Zod, Tailwind CSS, Recharts	Fast SPA, typed forms, cached server state.
API client	Generated from the FastAPI OpenAPI schema (`openapi-typescript`)	Front and back end cannot drift.
Testing	pytest + httpx (backend), Vitest + Testing Library (frontend), Playwright (end-to-end)	Unit, integration and E2E coverage.
Tooling	uv (Python deps), Ruff, mypy, ESLint, Prettier, pre-commit, GitHub Actions	Consistent quality gates.
Money	Integer cents (AUD); GST 10 % computed per line	No floating-point errors.

3. System architecture



- **Layered backend:** `routers` (HTTP, validation) → `services` (business rules, permission checks, transactions, audit) → `repositories/models` (SQLAlchemy). Routers never touch the DB directly.
- **Single origin:** SPA and API served under one domain (`app.example.com` and `app.example.com/api`) so session cookies stay first-party and `SameSite=Lax` works without CORS.
- **Stateless API processes:** sessions in Postgres, jobs in Redis, files in S3 → horizontal scaling.

4. Repository structure

```
crm-migration-agent/
├── backend/
│   ├── app/
│   │   ├── main.py              # FastAPI app, middleware, router registration
│   │   ├── core/                # config (pydantic-settings), security, logging, errors
│   │   ├── db/                  # engine/session, base model, mixins (timestamps, soft delete)
│   │   ├── models/              # SQLAlchemy models (one module per domain)
│   │   ├── schemas/             # Pydantic request/response models
│   │   ├── auth/                # login, sessions, MFA, password reset, dependencies
│   │   ├── permissions/         # role→permission matrix, record scopes, require() dependency
│   │   ├── services/            # leads, contacts, matters, billing, trust, documents, reports …
│   │   ├── routers/             # /api/v1/* endpoints
│   │   ├── storage/             # S3 + local adapters
│   │   ├── pdf/                 # WeasyPrint templates (invoice, agreement)
│   │   └── worker/              # Celery app, scheduled tasks, e-mail templates
│   ├── alembic/                 # migrations
│   ├── scripts/seed.py          # demo users (one per role), visa catalogue, sample data
│   ├── tests/                   # pytest: unit, API, permission matrix
│   └── pyproject.toml
├── frontend/
│   ├── src/
│   │   ├── api/                 # generated OpenAPI types + typed fetch client
│   │   ├── auth/                # session context, route guards, <Can permission=…>
│   │   ├── components/          # UI kit: DataTable, Path, RecordHeader, RelatedList, forms …
│   │   ├── features/            # leads, contacts, organisations, matters, tasks, billing, reports, admin
│   │   ├── portal/              # client portal pages
│   │   └── routes.tsx
│   ├── e2e/                     # Playwright tests
│   └── package.json
├── docker-compose.yml           # api, worker, beat, redis, minio (local S3), frontend
├── .github/workflows/ci.yml
└── docs/
```

5. Functional modules

Module	Features
Home dashboard	KPIs (open leads, active matters, lodged this month, outstanding invoices), my open tasks, deadlines due in 7/30 days, visa expiries, matter pipeline by stage, recent activity. Content adapts to role.
Leads	Capture (manual + public enquiry form endpoint with spam protection), list views with filters/search/sort/pagination, saved views, status Path, rating, owner, duplicate detection by e-mail/phone. **Convert** → Contact (+ optional Organisation) + optional Matter in one transaction, with lead history carried over.
Contacts (clients)	Identity, date of birth, citizenship, passport & expiry, current visa & expiry, onshore/offshore, family relationships, consents (privacy, marketing). Related: matters, documents, tasks, activities, invoices, client-account ledger, portal access.
Organisations	Employers / sponsors: ABN, industry, standard business sponsorship approval and expiry, related contacts and matters.
Matters	Auto-numbered `MAT-YYYY-NNNN`; visa subclass & stream; stage Path; status; priority; responsible RMA; case officer; applicants (primary, partner, dependants, sponsor, nominator); ImmiAccount TRN / application ID; lodgement date; RFI / natural-justice deadlines; decision date, outcome, grant number, granted visa expiry; fees. Document checklist generated from the visa template. Kanban board by stage.
Tasks & activities	Tasks (to-do, call, e-mail, meeting, follow-up, statutory deadline) with priority, due date, assignee and reminders. Activity timeline: notes, calls, e-mails, meetings, system events (stage changes, uploads, signatures).
Documents	Upload to matter/contact, category, client-visible flag, link to checklist item, versioning, virus scan hook, soft delete, pre-signed download links that expire in 5 minutes.
Billing	**Service agreements** (scope of services, professional fee, disbursements, government charges estimate, Consumer Guide provided flag, sent → signed by the client in the portal, PDF). **Invoices** (line items, GST / GST-free lines, draft → issued → paid / void, PDF, e-mail). **Payments**. **Client (trust) account** ledger per client: deposits, withdrawals, transfer-to-office against an invoice; balance can never go negative.
Client portal	Client login (invited by staff), view own matters and stage, outstanding checklist items, upload documents, download client-visible documents, secure messages with the team, sign service agreements, view/download invoices.
Reports	Lead funnel & conversion by source/owner, matters by stage / visa / outcome, grant rate, average processing time, revenue (invoiced, collected, outstanding, aged debtors 0–30/31–60/61–90/90+), client-account balances, staff workload. Date-range filters, charts, CSV export (audited).
Administration	Users (invite, role, activate/deactivate, reset MFA, unlock), visa catalogue and checklist templates, firm settings (name, ABN, MARA number, GST registration, invoice terms, bank & client-account details), audit log viewer with filters, data export.
Cross-cutting	Global search (leads, contacts, organisations, matters), in-app + e-mail notifications, scheduled reminders (task due, RFI deadline, visa expiry 90/30/7 days, sponsorship expiry), health checks.

Key workflows and states

Object	States
Lead status	NEW → CONTACTED → QUALIFIED → CONSULTATION_BOOKED → CONVERTED; or UNQUALIFIED / LOST (reason required). CONVERTED only via conversion.
Matter stage	CONSULTATION → AGREEMENT → DOCUMENT_COLLECTION → PREPARATION → READY_TO_LODGE → LODGED → FURTHER_INFO → DECISION. Status: OPEN / ON_HOLD / CLOSED. Outcome: GRANTED / REFUSED / WITHDRAWN / INVALID.

Matter rules	Moving past AGREEMENT requires a signed service agreement (admin can override with a reason, audited). Moving to LODGED, FURTHER_INFO or DECISION, and recording an outcome, requires `matter.lodge`. LODGED requires lodgement date and application reference. DECISION requires outcome and decision date.
Service agreement	DRAFT → SENT → SIGNED, or CANCELLED. Signed agreements are immutable.
Invoice	DRAFT → ISSUED → PARTIALLY_PAID → PAID, or VOID (reason required). Only drafts are editable; numbers allocated on issue.
Checklist item	PENDING → RECEIVED → APPROVED, or REJECTED (needs replacement) / NOT_APPLICABLE.

6. Roles

Role (code)	Typical user	Summary of access
`ADMIN`	Principal / system administrator	Full access including users, firm settings, audit log, deletes and stage overrides.
`PRACTICE_MANAGER`	Office / practice manager	All CRM data, billing, client account, reports, visa catalogue, audit log. Cannot manage users or firm settings, or delete matters.
`MIGRATION_AGENT`	Registered Migration Agent (RMA)	Full client work on all matters, including **lodgement and outcomes** (registered-agent-only actions), service agreements and invoices. Cannot manage the client account, record payments, or delete leads, contacts, organisations or matters.
`CASE_OFFICER`	Paralegal / case officer	Works only on matters **assigned** to them: documents, checklist, tasks, notes, client messages. Cannot lodge, record outcomes, or see billing.
`INTAKE_CONSULTANT`	Sales / intake	Own and unassigned leads, lead conversion, contacts and organisations. No matters, billing or reports.
`ACCOUNTS`	Finance officer	Invoices, payments, client account, billing reports; read-only contacts and matters.
`AUDITOR`	Compliance / external reviewer	Read-only access to everything including audit log and reports. Cannot change any data.
`CLIENT`	Agency client	Client portal only, limited to their own contact record, matters, documents, messages, agreements and invoices.

7. Permission matrix and access rules

✓ = granted for all records. **own** = only records the user owns or is assigned to (enforced in queries). Blank = denied. Permissions are checked on the server for every request; the UI only hides what the server would refuse.

Permission	ADMIN	PRAC. MGR	RMA	CASE OFF.	INTAKE	ACCOUNTS	AUDITOR	CLIENT
General								
dashboard.view	✓	✓	✓	✓	✓	✓	✓	
search.global	✓	✓	✓	own	own	✓	✓	
Leads								
lead.read	✓	✓	✓		own		✓	
lead.create / lead.update	✓	✓	✓		own			
lead.assign (change owner)	✓	✓	✓					
lead.convert	✓	✓	✓		own			
lead.delete	✓	✓						
Contacts & organisations								
contact.read	✓	✓	✓	✓	✓	✓	✓	own (portal profile)
contact.create / contact.update	✓	✓	✓	✓	✓			
contact.delete	✓	✓						
contact.portal_invite	✓	✓	✓	own				
organisation.read	✓	✓	✓	✓	✓	✓	✓	
organisation.create / update	✓	✓	✓		✓			
organisation.delete	✓	✓						
Matters								

Permission	ADMIN	PRAC. MGR	RMA	CASE OFF.	INTAKE	ACCOUNTS	AUDITOR	CLIENT
matter.read	✓	✓	✓	own		✓	✓	own
matter.create	✓	✓	✓					
matter.update (details, stage up to READY_TO_LODGE, checklist, applicants)	✓	✓	✓	own				
matter.assign (RMA / case officer)	✓	✓	✓					
matter.lodge (lodgement, RFI, decision, outcome)	✓	✓	✓					
matter.override (skip agreement gate)	✓							
matter.delete	✓							
Tasks, activities, documents, messages								
task.read	✓	✓	✓	own	own	own	✓	
task.create / task.update	✓	✓	✓	own	own	own		
task.delete	✓	✓	✓					
activity.create (log note/call/e-mail/meeting)	✓	✓	✓	own	own	✓		
document.read / download	✓	✓	✓	own			✓	own, client-visible
document.upload	✓	✓	✓	own				own
document.delete (soft)	✓	✓	✓					
message.read	✓	✓	✓	own			✓	own
message.send	✓	✓	✓	own				own
Billing & client account								
agreement.read	✓	✓	✓	own		✓	✓	own
agreement.manage (create, send, cancel)	✓	✓	✓					
agreement.sign								own
invoice.read	✓	✓	✓			✓	✓	own (issued)
invoice.manage (create, issue, void)	✓	✓	✓			✓		
payment.record	✓	✓				✓		
trust.read	✓	✓	✓			✓	✓	
trust.manage (deposit, withdraw, transfer)	✓	✓				✓		
Reports & administration								
report.view	✓	✓	✓			billing only	✓	
report.export (CSV)	✓	✓				billing only	✓	
visa.manage (catalogue, checklists)	✓	✓						
user.manage	✓							
settings.manage	✓							
audit.read	✓	✓					✓	
portal.access								✓

Definition of "own"

- **Matters (case officer):** user is the matter's case officer or responsible agent. Everything hanging off a matter (documents, checklist, messages, agreements, tasks on it) inherits this.
- **Matters (client):** the client's contact is the primary client or a listed applicant on the matter.
- **Leads (intake):** lead owner is the user, or the lead is unassigned (claiming sets the owner).
- **Tasks:** user is assignee or creator; a task on an accessible matter is also visible.
- **Activities / global search:** limited to records the user can read.

Additional access rules

- Staff cannot use portal endpoints and clients cannot use staff endpoints (separate router groups with separate dependencies).
- Only `ADMIN` may create or change another admin. Users cannot change their own role or deactivate themselves. The last active admin cannot be deactivated.
- A record outside the user's scope returns **404** (not 403) so its existence is not disclosed.
- Deletes are refused when dependent records exist (e.g. a contact with matters or invoices); most "deletes" are soft or replaced by deactivate/void.
- Issued invoices and signed agreements are immutable; corrections are made by voiding / new versions.
- Client-account withdrawals and transfers cannot exceed the client's balance (checked inside a locked transaction).
- Implementation: one Python module holds the matrix (`ROLE_PERMISSIONS`); a FastAPI dependency `require("matter.update")` guards each route; scope helpers add `WHERE` clauses; a parametrised pytest suite tests every role × endpoint against this table.

8. Data model

All tables have UUID primary keys, `created_at`, `updated_at`. Enumerations are PostgreSQL `CHECK`-constrained text columns mirrored as Python `StrEnum`s. Money is `BIGINT` cents.

Table	Key columns
users	email (unique, case-insensitive), name, password_hash, role, is_active, mfa_secret (encrypted), mara_number, contact_id (client users), failed_logins, locked_until, last_login_at
sessions	token_hash (unique), user_id, expires_at, last_seen_at, ip, user_agent, mfa_verified
password_reset_tokens / invitations	token_hash, user_id / contact_id, expires_at, used_at
firm_settings	firm_name, abn, mara_number, gst_registered, gst_rate_bp, invoice_terms_days, bank details, client-account details
sequences	key, value (matter, invoice, agreement numbering; row-locked increment)
organisations	name, abn, industry, contact details, is_sponsor, sponsor_approval_date, sponsor_expiry_date, owner_id
contacts	names, dob, gender, marital status, country of birth, citizenship, passport no./expiry, current visa/expiry, in_australia, address, occupation, consents, status, organisation_id, owner_id
contact_relationships	contact_id, related_contact_id, relationship (partner, child, parent, sibling)
leads	names, contact details, country, citizenship, visa_interest, source, status, rating, estimated_value, owner_id, lost_reason, converted_contact_id, converted_matter_id, converted_at
visa_types / visa_checklist_templates	subclass, stream, name, category, default fees, is_active / name, category, required, sort_order
matters	matter_number, title, client_id, organisation_id, visa_type_id, stage, status, priority, location, responsible_agent_id, case_officer_id, application_reference, lodged_at, decision_date, outcome, grant_number, granted_visa_expiry, fees, client_status_note, closed_at
matter_applicants	matter_id, contact_id, role (unique per matter + contact)
checklist_items	matter_id, name, category, required, status, client_visible, document_id, notes
tasks	subject, type, status, priority, due_at, completed_at, assignee_id, created_by_id, matter_id / contact_id / lead_id
activities	type, subject, body, occurred_at, user_id, polymorphic parent (matter / contact / lead / organisation)
documents	file_name, storage_key, mime_type, size, sha256, category, client_visible, uploaded_by_client, version, matter_id / contact_id, deleted_at
messages	matter_id, sender_id, from_client, body, read_at
service_agreements	number, matter_id, status, scope, professional_fee, disbursements, government_fee, payment_terms, consumer_guide_provided, sent_at, signed_at, signed_by_name, signed_ip, pdf_document_id
invoices / invoice_lines	number, status, contact_id, matter_id, issue/due dates, subtotal, gst, total, amount_paid, void_reason / description, qty, unit_price, gst_applicable, amount, gst
payments	invoice_id, amount, method, reference, received_at, trust_transaction_id
trust_transactions	contact_id, matter_id, invoice_id, type (DEPOSIT / WITHDRAWAL / TRANSFER_TO_OFFICE), amount, description, reference, transaction_date — append-only; reversals are new entries
notifications	user_id, title, body, link, read_at
audit_logs	user_id, action, entity_type, entity_id, summary, changes (JSONB before/after), ip, user_agent, request_id — append-only (DB role has no UPDATE/DELETE)

9. API design

- REST under `/api/v1`; OpenAPI at `/api/docs` (disabled in production or admin-only).
- List endpoints: `?page=&page_size=&sort=&q=` plus field filters; response `{ items, total, page, page_size }`.
- Errors: `{ "error": { "code", "message", "fields": {…} } }`; 401 unauthenticated, 403 missing permission, 404 not found / out of scope, 409 state conflict, 422 validation.
- Optimistic concurrency: updates send `updated_at`; mismatch → 409 so two staff cannot overwrite each other.
- Idempotency keys on payment and trust endpoints.

Group	Main endpoints
Auth	POST /auth/login, /auth/mfa/verify, /auth/logout, /auth/password/forgot, /auth/password/reset, /auth/invitation/accept · GET /auth/me (user + permissions)
Leads	CRUD /leads · POST /leads/{id}/convert · POST /leads/public-enquiry
Contacts / Organisations	CRUD /contacts, /organisations · POST /contacts/{id}/portal-invite · GET /contacts/{id}/trust-ledger
Matters	CRUD /matters · POST /matters/{id}/stage · /lodge · /decision · /close · /reopen · applicants, checklist, timeline sub-resources · GET /matters/board
Tasks / Activities	CRUD /tasks · POST /tasks/{id}/complete · POST /activities
Documents	POST /documents (multipart) · GET /documents/{id}/download (pre-signed URL) · DELETE
Billing	/agreements (+ /send, /cancel, /pdf) · /invoices (+ /issue, /void, /pdf, /send) · /payments · /trust-transactions
Portal	/portal/matters, /portal/documents, /portal/messages, /portal/agreements/{id}/sign, /portal/invoices
Reports / Admin	/reports/{name}?from&to (+ .csv) · /search · /notifications · /admin/users · /admin/visa-types · /admin/settings · /admin/audit-logs

10. Frontend design

- **Salesforce-style shell:** top bar (app switcher, global search, notifications, user menu), object tabs (Home, Leads, Contacts, Organisations, Matters, Tasks, Billing, Reports, Admin) shown only when the user has the read permission.
- **List views:** reusable `DataTable` with column sort, filters, search, pagination, saved views, bulk actions (assign owner, export).
- **Record pages:** highlights panel, stage `Path` component with "Mark stage complete", details tab (inline edit), related lists, activity timeline with composer (log call / note / e-mail / task).
- **Matter board:** Kanban by stage; drag-and-drop calls the same stage endpoint (server rules still apply).
- **Permission-aware UI:** `/auth/me` returns the permission list; `<Can permission="invoice.manage">` hides controls; route guards redirect; server remains the authority.
- **Client portal:** separate, simplified layout under `/portal`, mobile-first.
- **Accessibility & UX:** WCAG 2.1 AA, keyboard navigation, focus states, form error summaries, toasts, loading skeletons, empty states, responsive down to 360 px, Australian date (DD/MM/YYYY) and AUD formats.

11. Security

Control	Implementation
Authentication	Argon2id password hashes; password policy (≥ 12 chars, breached-password check via k-anonymity API optional); lockout after 5 failures for 15 minutes; mandatory TOTP MFA for staff, optional for clients; password reset by single-use 30-minute token.
Sessions	Opaque 256-bit token in `HttpOnly`, `Secure`, `SameSite=Lax` cookie; only SHA-256 hash stored; 12 h absolute / 60 min idle timeout; revoked on logout, password change, deactivation; admins can revoke all sessions of a user.
CSRF	Double-submit token header (`X-CSRF-Token`) required on all state-changing requests, plus Origin check.
Authorisation	Deny-by-default dependency on every route; record scopes in queries; automated matrix tests.
Input / output	Pydantic validation, SQLAlchemy parameterised queries, React escaping, strict Content-Security-Policy, HSTS, X-Frame-Options DENY, nosniff.
Files	Size limit 20 MB, MIME sniffing + extension allow-list (PDF, images, Office docs), random storage keys, S3 server-side encryption, private bucket, short-lived pre-signed URLs, malware-scan hook (ClamAV) before release.
Data protection	TLS everywhere (Neon requires SSL); passport numbers and MFA secrets encrypted at application level (Fernet key in secrets manager); backups via Neon point-in-time restore.
Abuse	Rate limiting (login, password reset, public enquiry form, uploads) via Redis; CAPTCHA on public form.
Audit & monitoring	Append-only audit log of create/update/delete/login/export/download/sign; structured JSON logs with request IDs; Sentry error tracking; uptime health checks.
Secrets & supply chain	Environment variables from a secrets manager, none in Git; Dependabot, `pip-audit`, `npm audit`, secret scanning in CI.

12. Australian compliance support

> The CRM supports compliance workflows; it does not by itself guarantee compliance. Requirements below should be confirmed by the principal against current OMARA guidance before go-live.

- **Code of Conduct for registered migration agents:** service agreement record with scope, fees and estimated charges; flag that the Consumer Guide was provided; agreement gate before work progresses; client sign-off captured with name, time and IP.
- **Client money:** separate client (trust) account ledger per client, no overdraw, transfer-to-office only against an issued invoice, full statement per client, reconciliation report.
- **Record keeping:** documents, communications and billing retained; soft deletes; retention period configurable (default 7 years) with purge job requiring admin approval.
- **Registered-agent actions:** lodgement and outcome recording restricted to `matter.lodge`; RMA MARA number stored on the user and shown on agreements and invoices.
- **Privacy Act 1988 / APPs:** privacy and marketing consent fields, data minimisation, access logging, export of a client's data on request, data hosted in Australia (Neon Sydney, S3 Sydney).
- **GST:** professional fees taxable at 10 % when the firm is GST-registered; government visa application charges passed through as GST-free lines; tax-invoice fields (ABN, "Tax Invoice" title) on PDFs.

13. Neon database operations

- **Region:** create the Neon project in AWS Asia Pacific (Sydney) `aws-ap-southeast-2`. The region is fixed at project creation.
- **Two connection strings:** `DATABASE_URL` = pooled endpoint (`-pooler` host, PgBouncer transaction mode) for the API and workers; `DATABASE_URL_DIRECT` = direct endpoint for Alembic migrations and long-running admin scripts.
- **Driver settings:** psycopg 3 with `sslmode=require`. On the pooled endpoint use client-library (protocol-level) prepared statements only, never SQL `PREPARE`; if the deployed libpq is older than PostgreSQL 17, disable psycopg prepared statements (`prepare_threshold=None`) to stay safe. Avoid session-level features (`LISTEN/NOTIFY`, session advisory locks, `SET` without `LOCAL`) on pooled connections.
- **Pooling in the app:** small SQLAlchemy pool (`pool_size=5`, `pool_pre_ping=True`, `pool_recycle=300`) because Neon can scale compute to zero and drop idle connections.
- **Branches:** `main` (production), `staging`, and a short-lived branch per pull request created by CI for migration and E2E tests, deleted on merge.
- **Backups / restore:** Neon point-in-time restore window sized for the plan; nightly logical dump (`pg_dump`) to S3 for long-term retention.
- **Cold starts:** keep production compute always-on (disable scale-to-zero) to avoid first-request latency; allow scale-to-zero on dev branches.

14. Testing and quality

Layer	What is tested
Backend unit	GST and invoice totals, trust balance rules, stage transition rules, numbering, permission matrix (every role × permission), scope helpers.
Backend API	pytest + httpx against a disposable Postgres (Neon CI branch or local container): each endpoint for allowed role, denied role, out-of-scope record, validation errors.
Frontend	Vitest + Testing Library for components (DataTable, Path, forms, <Can>).
End-to-end	Playwright: login + MFA, lead → conversion → matter, checklist + upload, agreement send → portal sign, invoice issue → payment / trust transfer, report CSV, role restrictions.
Gates in CI	Ruff, mypy (strict), ESLint, `tsc`, all tests, Alembic upgrade/downgrade check, dependency audit. Target ≥ 85 % backend coverage on services.
Non-functional	Load test (k6) for list/search endpoints: p95 < 300 ms at 50 concurrent users; OWASP ZAP baseline scan; accessibility audit (axe).

15. Deployment and environments

- **Local:** `docker compose up` runs API, worker, beat, Redis, MinIO (local S3) and the Vite dev server; database is a personal Neon dev branch (or local Postgres). `scripts/seed.py` creates one demo user per role.
- **Environments:** development → staging → production, each with its own Neon branch/project, S3 bucket and secrets.
- **Containers:** one Python image (API / worker / beat via command), one static frontend build served by CDN or Nginx.
- **Hosting (recommended):** AWS Sydney (ECS Fargate or App Runner) to keep data in Australia, close to Neon `aws-ap-southeast-2`.
- **CI/CD (GitHub Actions):** lint → test → build images → run `alembic upgrade head` on the direct endpoint → deploy → smoke test. Production deploys need manual approval.

16. Delivery phases

#	Phase	Deliverables	Estimate
0	Foundations	Repos, tooling, CI, Neon project + branches, Docker Compose, config, logging, error format.	1 week
1	Auth & RBAC	Users, sessions, MFA, password reset, invitations, permission matrix + scopes + tests, audit log, app shell & UI kit.	2 weeks
2	Core CRM	Leads (+ conversion), contacts, organisations, global search, activities, tasks, notifications.	2–3 weeks
3	Matters	Visa catalogue, matters, Path + rules, applicants, checklist, documents (S3), board view.	2–3 weeks
4	Billing	Service agreements + PDF, invoices + PDF, payments, client-account ledger.	2 weeks
5	Client portal	Invitations, portal pages, uploads, messages, agreement signing, invoices.	1–2 weeks
6	Reports & dashboard	Dashboards per role, reports, charts, CSV export, scheduled reminder jobs.	1–2 weeks
7	Admin & hardening	Admin screens, retention job, security review, load test, accessibility, E2E suite, documentation, UAT, go-live.	2 weeks

Estimates assume a small team (1 backend, 1 frontend developer, part-time QA). Each phase ends with a demo and passing CI.

17. Risks and open questions

Item	Mitigation / decision needed
Government fees and visa streams change (usually each 1 July)	Visa catalogue and fees are admin-editable data, not code.
Neon scale-to-zero latency and pooler limits	Always-on compute in production; pooled endpoint; settings in §13.
No ImmiAccount API	Manual entry of TRN, lodgement and decision data; documents uploaded manually.
E-signature legal weight	Typed-name + checkbox + timestamp + IP in v1; confirm with principal whether a third-party e-sign provider (e.g. DocuSign) is required.
Open questions	(1) Hosting provider and domain. (2) E-mail provider (SES, SendGrid, M365). (3) Need for Xero/MYOB sync. (4) Number of users and offices (affects multi-office support). (5) Branding.